

Projektdokumentation

Entwicklung einer Software zur Lagerverwaltung von medizinischem Verbrauchsmaterial in einem Krankenhaus

Verfasser	Sascha Schulz
Gruppe	2551
Ausbildungsberuf	Wirtschaftsinformatiker (WI)
Ausbildungsstätte	SRH Berufliche Rehabilitation GmbH
Betreuerin	Frau Cramer
Projektzeitraum	04.05.2026 – 17.07.2026

Inhaltsverzeichnis

1 Einleitung	5
1.1 Projektbeschreibung	5
1.2 Projektziel	5
1.3 Projektumfeld	5
1.4 Projektbegründung.....	6
1.5 Ansprechpartner.....	6
2 Projektplanung	6
2.1 Projektphasen.....	6
2.2 Ressourcenplanung.....	7
2.3 Risikomanagement	7
2.4 Entwicklungsprozess	8
3 Analysephase	8
3.1 Ist-Analyse.....	8
3.2 Anwendungsfälle	8
3.3 Lastenheft	9
4 Entwurfsphase	9
4.1 Datenbankentwurf	9
4.2 Architekturentwurf (MVC + DAO).....	10
4.3 Entwurf der Benutzeroberfläche.....	11
4.4 Pflichtenheft.....	12
5 Implementierungsphase	12
5.1 Datenbank und JDBC-Verbindung.....	12
5.2 DAO-Schicht und Model-Klassen.....	13
5.3 Benutzeroberfläche und Löschen	14
5.4 Such- und Filterfunktion.....	15
5.5 Mindestbestand-Warnung und Lagerbuchungen.....	15
6 Abnahme- und Testphase	16
6.1 Testfälle und Testergebnisse	16
6.2 Soll-Ist-Vergleich	17
6.3 Abnahme und Übergabe	17
7 Dokumentation	17
7.1 Benutzerhandbuch	17
7.2 Entwicklerdokumentation.....	18
8 Fazit	18
8.1 Soll-Ist-Vergleich der Projektziele	18
8.2 Lessons Learned	19
8.3 Ausblick.....	20

Abgabeerklärung	21
Quellenverzeichnis	22
Literaturquellen	22
Internetquellen.....	22
Anhang	24
Anhang A – ER-Diagramm	24
Anhang B – SQL-Datenbankschema.....	25
Anhang C – Benutzerhandbuch.....	27
Anhang D – Wochenplan	31
Anhang E – Projektantrag.....	31
Anhang F – Glossar	32
Anhang G – Lastenheft.....	35
Anhang H – Pflichtenheft.....	37

Abbildungsverzeichnis

Abbildung 1: Use-Case-Diagramm der Hauptfunktionen	9
Abbildung 2: Schichtenarchitektur (MVC mit DAO-Schicht).....	11
Abbildung 3: Entwurf des Hauptfensters (Bestandsübersicht).....	11
Abbildung 4: Programmablaufplan „Material löschen mit Sicherheitsabfrage“	14
Abbildung 5: ER-Diagramm der Datenbank (Anhang A).....	24

Tabellenverzeichnis

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen.....	5
Tabelle 2: Projektphasen und Stundenverteilung	6
Tabelle 3: Kostenkalkulation des Projekts (fiktiv)	7
Tabelle 4: Risiken und Gegenmaßnahmen	8
Tabelle 5: Datenbanktabellen im Überblick	10
Tabelle 6: Zuordnung der Klassen zu den MVC-Schichten.....	14
Tabelle 7: Definierte Testfälle	16
Tabelle 8: Soll-Ist-Vergleich der Anforderungen.....	17
Tabelle 9: Soll-Ist-Vergleich der Projektstunden	19
Tabelle 10: Systemvoraussetzungen.....	27

Tabelle 11: Registerkarten und ihre Funktionen	28
Tabelle 12: Häufige Fragen und Fehlerbehebung	29
Tabelle 13: Detaillierte Aufschlüsselung der Projektphasen.....	31
Tabelle 14: Glossar der Fachbegriffe	32
Tabelle 15: Pflichtanforderungen laut Ausbildungsrichtlinie	35
Tabelle 16: Krankenhausspezifische Zusatzanforderungen	36
Tabelle 17: Zuordnung der Anforderungen zur technischen Umsetzung.....	38

Quellcodeverzeichnis

Listing 1: CREATE-Statement der Tabelle materialien.....	12
Listing 2: Singleton-Datenbankverbindung (DBConnection).....	13
Listing 3: Löschen: delete() in MaterialDAO.....	13
Listing 4: Gemeinsame Schnittstelle FindDeleteDAO<T>	13
Listing 5: Filterung der Bestandsübersicht nach Namen und Station	15
Listing 6: Mindestbestand-Warnung über eine RowFactory.....	15
Listing 7: Datenbankschema – CREATE-Statements aller sechs Tabellen	26
Listing 8: Start-Skript start_lagerverwaltung.bat	27

1 Einleitung

1.1 Projektbeschreibung

In der DOIT-Projektphase der SRH Beruflichen Rehabilitation GmbH [\[4\]](#) entstand eine Desktop-Anwendung für die Lagerverwaltung von medizinischem Verbrauchsmaterial im Krankenhaus – etwa Verbandsmaterial, Spritzen oder Einmalhandschuhe. Umgesetzt wurde sie mit Java 17 und JavaFX für die Oberfläche, die Daten liegen in einer MySQL-Datenbank. Die Anwendung läuft lokal als Einzelplatzlösung und kommt ohne Internetanbindung aus.

Das Personal sieht damit jederzeit, welche Bestände auf den einzelnen Stationen vorhanden sind. Wird ein Material knapp, weist die Anwendung darauf hin; jede Ein- und Ausbuchung wird protokolliert.

1.2 Projektziel

Ziel war eine Anwendung, die sich ohne Einarbeitung bedienen lässt. Das Krankenhauspersonal soll damit Bestände einsehen, nicht mehr benötigte Einträge löschen, nach dem Materialnamen suchen und nach Station filtern können; sinkt ein Bestand unter den Mindestbestand, erscheint eine Warnung. Vorgegeben waren dazu eine MySQL-Datenbank mit mindestens fünf Tabellen, eine CRUD-Funktionalität (Create, Read, Update, Delete), eine MVC-Architektur (Model-View-Controller) sowie eine Such- bzw. Filterfunktion – diese vier Pflichtenforderungen bilden den Kern des Projekts. Wie weit das CRUD umgesetzt wurde, steht im Soll-Ist-Vergleich ([Kapitel 8.1](#)).

1.3 Projektumfeld

Das Projekt wurde als Einzelprojekt in der Ausbildung zum Wirtschaftsinformatiker (WI) durchgeführt. Betreut und abgenommen wird es von Frau Cramer. Inhaltlich orientiert es sich an einem realistischen Fall aus dem Krankenhausalltag. Auftraggeber und Endnutzer – Krankenhausleitung sowie Pflege- und Stationspersonal – sind fiktiv und dienen nur der praxisnahen Darstellung.

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen

Rolle	Person	Funktion
Betreuerin	Frau Cramer	Fachliche Begleitung, Abnahme
Fiktiver Auftraggeber	Krankenhausleitung	Anforderungssteller für die Software
Fiktive Endnutzer	Pflege- und Stationspersonal	Tägliche Nutzung der Materialverwaltung

1.4 Projektbegründung

In vielen Häusern wird Verbrauchsmaterial bis heute von Hand in Listen oder Tabellen gepflegt. Das kostet Zeit und ist fehleranfällig. Wer wissen will, wie viel auf einer Station noch liegt, muss erst die Liste durchgehen; eine Warnung bei knappem Material gibt es nicht, und im Nachhinein lässt sich kaum nachvollziehen, wer was entnommen hat. Genau hier setzt die Anwendung an. Sie verringert Fehlbestände, macht Material schneller auffindbar und gibt der Verwaltung eine Historie, die sich jederzeit prüfen lässt.

1.5 Ansprechpartner

Während des Projekts war Frau Cramer die zentrale Ansprechpartnerin. An sie konnte sich der Autor bei fachlichen Fragen zur Umsetzung genauso wenden wie bei organisatorischen Fragen zum Ablauf.

2 Projektplanung

2.1 Projektphasen

Das Projekt gliedert sich in sieben Phasen mit einem geplanten Gesamtaufwand von 180 Stunden, verteilt auf sieben Projektwochen. Die Aufteilung folgt dem Projektantrag (v2) und dem SRH-Leitfaden.

Tabelle 2: Projektphasen und Stundenverteilung

#	Phase	Inhalt	h
1	Analyse & Planung	Anforderungen, ER-Diagramm, DB-Schema, Wochenplan	20
2	Datenbank	SQL-Skripte, 6 Tabellen, Testdaten, JDBC-Verbindung	20
3	DAO & Model	MVC-Architektur, Model-Klassen, DAO-Schicht über generisches Interface FindDeleteDAO<T>	20
4	UI-Grundfunktionen	Hauptfenster, Tabellen aller 6 Entitäten, generische Basisklasse EntityCrud<T>, Löschen, Dialoge	40
5	UI erweitert	Suche nach Namen, Filter nach Station, Mindestbestand-Warnung	25
6	Test & Bugfixes	Funktionen testen, Fehler beheben, Code-Review	25
7	Dokumentation	Benutzerhandbuch, Projektdoku, Abgabe	30
Σ	Gesamt		180

2.2 Ressourcenplanung

Als Einzelprojekt lagen Analyse, Entwicklung, Test und Dokumentation in einer Hand. An Sachmitteln genügte ein privater Laptop (Windows 10/11, ≥ 8 GB RAM); dazu kamen Java 17 [1], JavaFX 21 [2], MySQL 8 [3] mit Connector [11], IntelliJ IDEA, Maven [10] und Git – allesamt kostenfrei. Zur Übung der Kalkulation wurde zusätzlich eine fiktive Kostenrechnung aufgestellt: 150 Stunden Projektarbeit (alle Phasen außer der Dokumentation) zu einem angenommenen Satz von 80,00 €/h ergeben 12.000,00 €. Dazu kommen rund 150,00 € anteilige Sachkosten für Strom, Internet und die Abnutzung des Laptops. Als Gesamtpreis ergeben sich damit 12.150,00 €. In der Ausbildung fallen diese Kosten natürlich nicht an.

Tabelle 3: Kostenkalkulation des Projekts (fiktiv)

Position	Berechnung	Betrag
Personalkosten (alle Phasen außer Dokumentation)	150 h $\times$ 80,00 €/h	12.000,00 €
Anteilige Sachkosten (Strom, Internet, Abnutzung Laptop)	pauschal	150,00 €
Software und Werkzeuge (Java, JavaFX, MySQL, IntelliJ, Maven, Git)	kostenfrei	0,00 €
Gesamtpreis	Personal- + Sachkosten	12.150,00 €

Vor der Umsetzung stand die Frage, ob eine individuelle Lösung entwickelt oder eine Standardsoftware eingekauft wird (Make-or-Buy). Die Entscheidung fiel bewusst auf die Individualentwicklung. Eine Standardlösung hätte laufende Lizenzkosten verursacht und ließe sich nur schwer an die individuellen Abläufe im Krankenhaus anpassen. Bei der Individualentwicklung fallen keine Lizenzkosten an, und die Anwendung lässt sich genau auf die Anforderungen zuschneiden. Dazu kommt, dass die Individualentwicklung in der Ausbildung ohnehin das Ziel des Projekts ist.

2.3 Risikomanagement

Neben Kosten und Ressourcen wurden auch Risiken für den Projektverlauf eingeplant. Für die wichtigsten wurden frühzeitig Gegenmaßnahmen getroffen.

Tabelle 4: Risiken und Gegenmaßnahmen

Risiko	Wahrsch.	Gegenmaßnahme
JDBC-Verbindungsprobleme	Mittel	Früher Test, Singleton-DBConnection
JavaFX-Komplexität (TableView)	Hoch	Einfache Layouts zuerst, schrittweise erweitern
Zeitverzug in Phase 4	Mittel	12 h Puffer, Kernanforderungen zuerst

2.4 Entwicklungsprozess

Entwickelt wurde schrittweise entlang der Phasen. Den Anfang machten das Datenmodell und die Datenzugriffsschicht, erst danach kam die Oberfläche dazu. Bei der Oberfläche wurden zunächst die Kernmodule Materialien, Kategorien und Stationslager fertiggestellt, die optionalen Module Lieferanten, Bestellungen und Bestandsbewegungen folgten später. Auf diese Weise stand früh ein lauffähiger Kern, an dem sich die Pflichtenanforderungen überprüfen ließen.

3 Analysephase

3.1 Ist-Analyse

Ausgangspunkt ist die manuelle Materialverwaltung per Liste oder Tabellenkalkulation. Wer den Bestand einer einzelnen Station braucht, zum Beispiel Station 1, OP, Intensiv oder Zentrallager, muss ihn sich erst zusammensuchen. Wird ein Bestand knapp, fällt das oft erst auf, wenn jemand die Liste durchgeht; wer eine Entnahme im Nachhinein nachvollziehen will, findet dafür keine feste Aufzeichnung. Bei der Pflege schleichen sich leicht Fehler ein, und gezielt nach Station oder Materialart filtern lässt sich ohnehin nicht.

3.2 Anwendungsfälle

Aus der Ist-Analyse ergaben sich die zentralen Anwendungsfälle. In der Bestandsübersicht sieht das Personal den aktuellen Bestand je Material und Lager. Es kann nach dem Materialnamen suchen und nach Station filtern. Sinkt ein Bestand unter den Mindestbestand, weist eine Warnung darauf hin. Nicht mehr benötigte Materialien und Kategorien lassen sich zusätzlich löschen, allerdings erst, wenn kein Bestand und keine Bestellung mehr dazu vorhanden sind. Bestellungen und Bestandsbewegungen lassen sich einsehen. Eine Entnahme wird als Ausgang gebucht, eine Umlagerung zwischen zwei Lagern als Ausgang im einen und Eingang im anderen Lager. Das folgende Use-Case-Diagramm ([Abbildung 1](#)) fasst diese Anwendungsfälle zusammen.

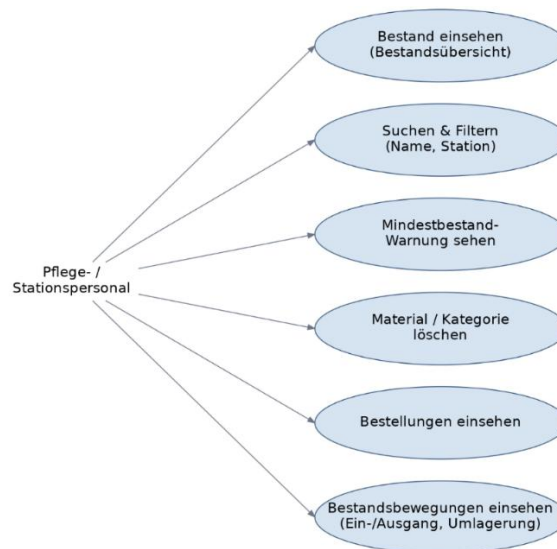


Abbildung 1: Use-Case-Diagramm der Hauptfunktionen

3.3 Lastenheft

Die Pflichtanforderungen ergeben sich aus der Ausbildungsrichtlinie: eine MySQL-Datenbank mit mindestens fünf Tabellen, CRUD-Funktionalität, eine MVC-Architektur sowie Such- und Filterfunktionen. Dazu kommen Zusatzanforderungen aus dem Krankenhauskontext: eine Mindestbestand-Warnung, eine lückenlose Buchungshistorie, ein Stationsfilter und eine Oberfläche, die ohne Einarbeitung bedienbar ist.

Die CRUD-Anforderung wurde auf Lesen und Löschen begrenzt; die frei gewordene Zeit floss in eine generische, leicht erweiterbare DAO-Basis (siehe [Kapitel 4.2](#) und [8.2](#)).

Das vollständige Lastenheft mit Ausgangssituation, Zielen, allen Anforderungen und den Abnahmekriterien befindet sich in [Anhang G](#).

4 Entwurfsphase

4.1 Datenbankentwurf

Auf Basis der Anforderungen wurde das Datenmodell in sechs normalisierten Tabellen (3. Normalform) entworfen. Das vollständige ER-Diagramm (Entity-Relationship-Diagramm) mit allen Entitäten, Attributen, Primär- und Fremdschlüsseln liegt in [Anhang A](#) vor. Der aktuelle Bestand steht nicht als Feld im Material, sondern wird aus den Bewegungen berechnet, Summe der Eingänge minus Summe der Ausgänge. Dadurch kann er nicht von der tatsächlichen Buchungslage abweichen, und die Historie bleibt vollständig nachvollziehbar.

Tabelle 5: Datenbanktabellen im Überblick

Tabelle	Zweck
kategorien	Materialgruppen (z. B. Verband, Injektion)
lieferanten	Medizinische Lieferanten mit Kontaktdaten
stationslager	Stationen und Lagerorte im Krankenhaus
materialien	Stammdaten aller Materialien inkl. Mindestbestand
bestellungen	Bestellhistorie mit Status und Lieferdatum
bestandsbewegungen	Buchungshistorie (EINGANG/AUSGANG); Bestand daraus berechnet

4.2 Architekturentwurf (MVC + DAO)

Die Anwendung basiert auf dem MVC-Muster, ergänzt durch eine DAO-Schicht (Data Access Object). Dabei zeigt die View (JavaFX/FXML, FX Markup Language) [\[9\]](#) Daten an, der Controller steuert die Logik und die DAO-Klassen kapseln sämtliche SQL-Zugriffe (Structured Query Language), während das Model als reine Datenstruktur (POJO, Plain Old Java Object) fungiert. Diese klare Schichtentrennung sorgt für eine übersichtliche Struktur und erleichtert künftige Anpassungen oder Erweiterungen.

Um Redundanzen zu vermeiden, nutzen alle DAO-Klassen das zentrale Interface FindDeleteDAO. Durch den Einsatz von Java-Generics (<T>) wird der Vertrag einmalig definiert und erst bei der Implementierung mit der jeweiligen Modellklasse verknüpft. Der Aufbau ist gezielt darauf ausgerichtet, künftige Erweiterungen ohne strukturelle Anpassungen zu ermöglichen: Neue Operationen, wie etwa das Anlegen von Datensätzen, müssen lediglich im Interface ergänzt und in den DAOs ausprogrammiert werden, ohne die Grundstruktur zu verändern.

Auch die Controller-Logik ist klar aufgeteilt. Der MainController ist nur noch Verteiler – er nimmt die Aktionen der Oberfläche entgegen und gibt sie an die zuständige Klasse weiter. Wiederkehrende Abläufe beim Laden und Löschen sind in der Basisklasse EntityCrud<T> gebündelt, was den MainController schlank hält und den Code wartbar macht. [Abbildung 2](#) zeigt diese Schichtenarchitektur im Überblick.

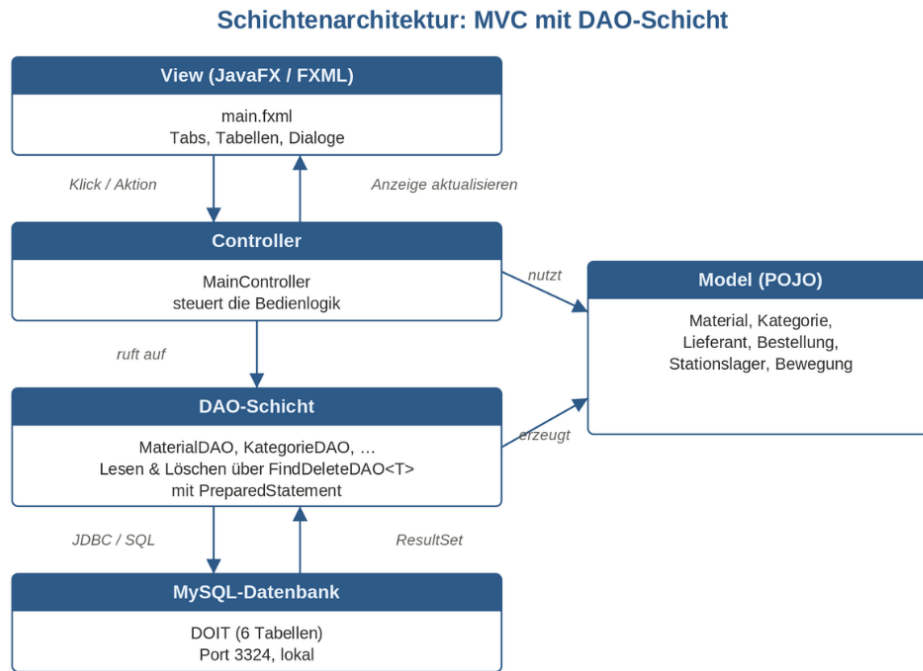


Abbildung 2: Schichtenarchitektur (MVC mit DAO-Schicht)

4.3 Entwurf der Benutzeroberfläche

Die Oberfläche besteht aus einem Hauptfenster mit sieben Registerkarten. Die erste Karte „Bestandsübersicht“ hat oben eine Such- und Filterleiste (Suchfeld für den Namen, Auswahl der Station) und darunter die Bestandstabelle; Zeilen mit zu niedrigem Bestand werden rot hervorgehoben. Die übrigen Karten (Materialien, Kategorien, Stationslager, Lieferanten, Bestellungen und Bestandsbewegungen) zeigen die jeweiligen Daten in einer Tabelle. Auf den Karten Materialien und Kategorien gibt es zusätzlich eine Schaltfläche „Löschen“

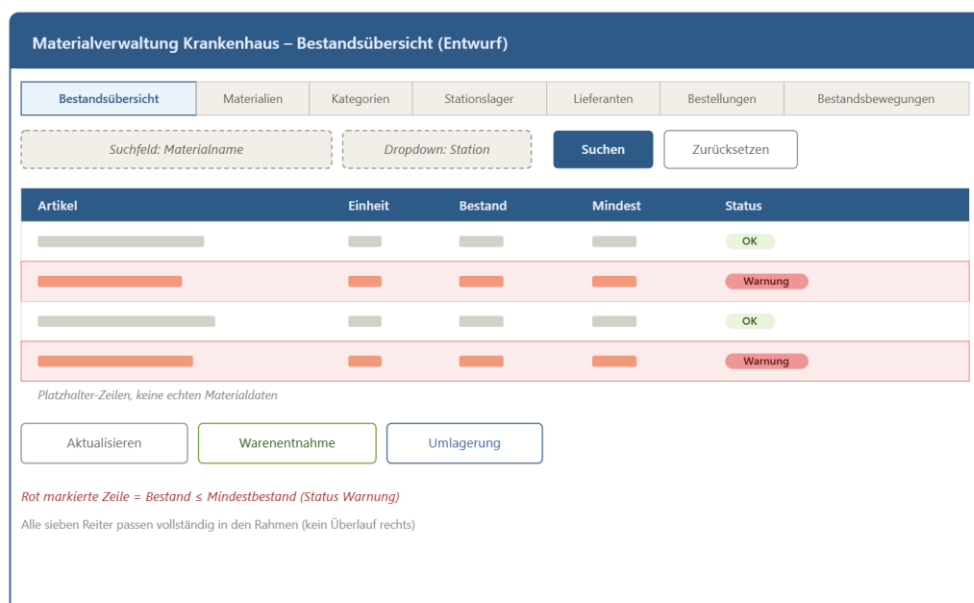


Abbildung 3: Entwurf des Hauptfensters (Bestandsübersicht)

4.4 Pflichtenheft

Die sechs Tabellen bilden das MySQL-Schema, die Lese- und Löschzugriffe laufen je Entität über PreparedStatements, und die Struktur folgt dem MVC-Muster mit DAO-Schicht. Die Bestandssicht wird nach Namen und Station gefiltert, die Mindestbestand-Warnung übernimmt eine RowFactory, die die betroffenen Zeilen einfärbt. Die Anwendung läuft lokal ohne Internetverbindung.

Das vollständige Pflichtenheft mit Zielbestimmung, Produktfunktionen, Datenmodell und der Zuordnung aller Anforderungen zur technischen Umsetzung befindet sich in [Anhang H](#). Bewusst nicht Teil des Pflichtenhefts sind das Anlegen und Bearbeiten von Datensätzen über die Oberfläche, eine Benutzerverwaltung mit Login, ein Betrieb im Netzwerk und Exportfunktionen. Die Reduktion auf Lesen und Löschen wurde bewusst gewählt.

5 Implementierungsphase

Dieses Kapitel zeigt die Umsetzung an einigen repräsentativen Code-Auszügen aus dem vollständigen Quellcode des Projekts.

5.1 Datenbank und JDBC-Verbindung

Das CREATE-Skript legt alle sechs Tabellen mit Primär- und Fremdschlüsseln an. Beispielfhaft die Haupttabelle materialien:

```
CREATE TABLE materialien (
  material_id    INT AUTO_INCREMENT PRIMARY KEY,
  name           VARCHAR(150) NOT NULL,
  einheit        VARCHAR(30)  NOT NULL,
  mindestbestand INT NOT NULL DEFAULT 0,
  kategorie_id   INT NOT NULL,
  FOREIGN KEY (kategorie_id)
    REFERENCES kategorien(kategorie_id)
);
```

Listing 1: CREATE-Statement der Tabelle materialien

Die Datenbankverbindung ist als Singleton umgesetzt, damit nur eine wiederverwendbare Verbindung besteht:

```
public class DBConnection {
  private static final String URL =
    "jdbc:mysql://127.0.0.1:3324/DOIT"
    + "?serverTimezone=Europe/Berlin";
  private static Connection connection;
  public static Connection getConnection() throws SQLException {
```

```

        if (connection == null || connection.isClosed())
            connection = DriverManager.getConnection(URL, "root",
"1234");
        return connection;
    }
}

```

Listing 2: Singleton-Datenbankverbindung (DBConnection)

5.2 DAO-Schicht und Model-Klassen

Jede DAO-Klasse regelt den Datenbankzugriff für ihre Tabelle. Lesende Zugriffe liefern Listen über `findAll()`; das Löschen nutzt ein `PreparedStatement` [\[12\]](#) gegen SQL-Injection [\[14\]](#). Beispiel für das Löschen eines Materials:

```

public void delete(int id) throws SQLException {
    String sql = "DELETE FROM materialien WHERE material_id = ?";
    Connection verbindung = DBConnection.getConnection();
    try (PreparedStatement anweisung =
verbindung.prepareStatement(sql)) {
        anweisung.setInt(1, id);
        anweisung.executeUpdate();
    }
}

```

Listing 3: Löschen: delete() in MaterialDAO

Damit der Vertrag nicht in jeder Klasse neu beschrieben werden muss, teilen sich alle DAO-Klassen die gemeinsame Schnittstelle `FindDeleteDAO<T>`. Das Interface enthält nur die beiden Methodenköpfe `findAll` und `delete`. Wer es implementiert, muss beide ausprogrammieren. Fehlt eine, verweigert der Compiler die Übersetzung. Der Platzhalter `<T>` kommt aus den Java-Generics. Er steht immer für einen Referenztyp, also ein Objekt wie `Material` oder `Kategorie`, und wird beim Implementieren festgelegt. `delete(int id)` nutzt dagegen den Wertetyp `int`, `findAll()` gibt seine Ergebnisse als `List<T>` zurück. So treffen an einer Stelle Wertetyp, Referenztyp und `Collection` zusammen. Diese drei Konzepte tauchen sonst im Code verstreut auf.

```

public interface FindDeleteDAO<T> {
    List<T> findAll()        throws SQLException;
    void    delete(int id)  throws SQLException;
}

```

Listing 4: Gemeinsame Schnittstelle FindDeleteDAO<T>

In den sechs DAO-Klassen kennzeichnet die Annotation `@Override` die beiden Methoden als Umsetzung des Schnittstellen-Vertrags; der Compiler prüft dadurch, dass die Signaturen exakt zur Schnittstelle passen.

Tabelle 6: Zuordnung der Klassen zu den MVC-Schichten

Schicht	Inhalt
Model	Material, Kategorie, Lieferant, Stationslager, Bestellung, Bestandsbewegung
View	JavaFX-FXML (main.fxml): Hauptfenster mit Registerkarten je Entität
Controller	MainController: steuert die UI-Logik, delegiert an die DAO-Schicht
DAO	MaterialDAO, KategorieDAO, LieferantDAO, StationslagerDAO, ...

5.3 Benutzeroberfläche und Löschen

Das Hauptfenster baut auf einem TabPane auf [7]. Die Bestandsübersicht zeigt für jedes Material und Lager den aktuellen Bestand. Dieser wird nicht gespeichert, sondern per JOIN über findBestandViews aus den Bewegungen berechnet. Die Tabelle hat die Spalten Material, Einheit, Lager, Bestand, Mindestbestand und Status. Die Registerkarte Materialien zeigt die Stammdaten: Name, Einheit, Mindestbestand und Kategorie-ID. Nicht mehr benötigte Einträge lassen sich dort und auf der Registerkarte Kategorien über die Schaltfläche Löschen entfernen. Vorher erscheint eine Sicherheitsabfrage [15]. Anlegen und Bearbeiten wurden aus Zeitgründen nicht mehr umgesetzt (siehe Kapitel 8.2).

Wie das Löschen mit der Sicherheitsabfrage abläuft, zeigt der folgende Programmablaufplan.

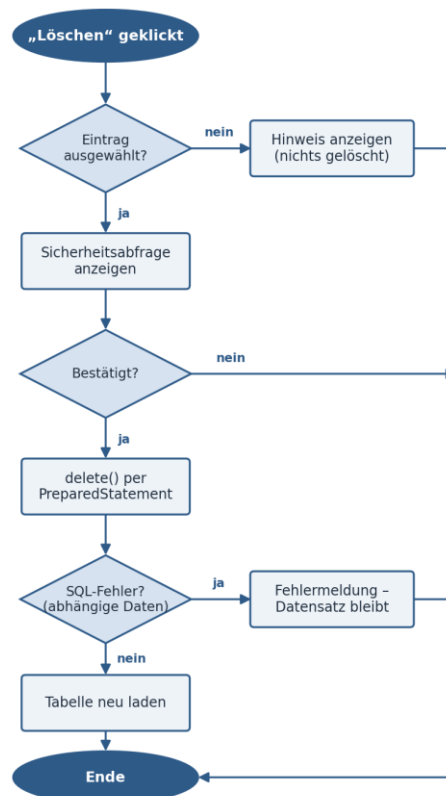


Abbildung 4: Programmablaufplan „Material löschen mit Sicherheitsabfrage“

5.4 Such- und Filterfunktion

Die Bestandsübersicht lässt sich filtern. Suchfeld (Materialname) und Stationsauswahl wirken zusammen, angezeigt wird nur, was beide Bedingungen erfüllt. Ausgelöst wird die Filterung über die Eingabetaste oder die Schaltfläche „Suchen“:

```
private void anzeigeFiltern() {
    if (alleBestaende == null) return;
    String suchtext = txtSuche.getText() == null
        ? "" : txtSuche.getText().trim().toLowerCase();
    Stationslager gewaehltesLager = comboBoxLager.getValue();
    ObservableList<BestandView> gefiltert =
        FXCollections.observableArrayList();
    int warnungen = 0;
    for (BestandView v : alleBestaende) {
        boolean passtName = suchtext.isEmpty()
            || v.getMaterialName().toLowerCase().contains(suchtext);
        boolean passtLager = gewaehltesLager == null
            || v.getLagerId() == gewaehltesLager.getId();
        if (passtName && passtLager) {
            gefiltert.add(v);
            if (v.isWarnung()) warnungen++;
        }
    }
    bestandListe.setAll(gefiltert);
    lblWarnungAnzahl.setText("Warnungen: " + warnungen);
}
```

Listing 5: Filterung der Bestandsübersicht nach Namen und Station

5.5 Mindestbestand-Warnung und Lagerbuchungen

Liegt der Bestand auf oder unter dem Mindestbestand, färbt eine RowFactory die Zeile rot:

```
tabBestand.setRowFactory(tv -> new TableRow<BestandView>() {
    @Override protected void updateItem(BestandView item,
                                         boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null)        setStyle("");
        else if (item.isWarnung())
            setStyle("-fx-background-color: #FADBD8;");
        else setStyle("");
    }
}); // isWarnung(): bestand <= mindestbestand
```

Listing 6: Mindestbestand-Warnung über eine RowFactory

Bestandsänderungen sind in der Datenbank als Buchungen hinterlegt. Auf der Registerkarte Bestandsbewegungen werden sie angezeigt: eine Entnahme als AUSGANG, eine Umlagerung zwischen zwei Lagern als AUSGANG im Quelllager und EINGANG im Ziellager. Der Status

einer Bestellung – offen, geliefert oder storniert – beschreibt nur den Bestellvorgang selbst; der zugehörige Wareneingang ist als EINGANG gebucht.

6 Abnahme- und Testphase

6.1 Testfälle und Testergebnisse

Zur Qualitätssicherung wurden neun Testfälle definiert, die alle Pflicht- und Zusatzanforderungen abdecken. Jeder davon wird manuell durchgeführt und das Ergebnis als bestanden oder fehlgeschlagen protokolliert. Da die Funktionen vollständig umgesetzt und die in [Phase 6](#) gefundenen Fehler behoben sind, haben alle Pflicht-Testfälle bestanden.

Tabelle 7: Definierte Testfälle

#	Testfall	Erwartetes Ergebnis	Prio	Ergebnis
T1	Materialien anzeigen	Tabelle zeigt alle Materialien aus der DB	Hoch	bestanden
T2	Kategorie löschen	Aus Tabelle und DB entfernt	Hoch	bestanden
T3	Material löschen	Aus Tabelle und DB entfernt	Hoch	bestanden
T4	Mindestbestand-Warnung	Zeile rot bei Bestand $\leq$ Mindestbestand	Hoch	bestanden
T5	Suche nach Name	Tabelle filtert nach Enter oder Klick auf Suchen	Mittel	bestanden
T6	Filter nach Station	Nur Material der gewählten Station	Mittel	bestanden
T7	Löschen mit abhängigen Daten	Fehlermeldung, Datensatz bleibt erhalten	Hoch	bestanden
T8	Bestandsübersicht anzeigen	Berechneter Bestand je Material und Lager	Mittel	bestanden
T9	Löschen ohne Auswahl	Hinweis erscheint, nichts wird gelöscht	Hoch	bestanden

Während der Umsetzung traten mehrere Fehler auf. Zwei davon stehen hier stellvertretend. In der BestandsbewegungDAO war ein Primärschlüssel falsch benannt. Das fiel direkt über die Fehlermeldung auf und war schnell behoben. Das zweite Problem lag bei der Datenbankverbindung. Sie schlug fehl, obwohl der Code stimmte. Erst beim Nachforschen zeigte sich die Ursache: Ein aktiver VPN-Client (Virtual Private Network) leitete den Netzwerkverkehr um, sodass 127.0.0.1:3324 nicht mehr auf den lokalen Docker-Container zeigte. Nach einer Ausnahmeregelung für diese IP lief die Verbindung wieder normal.

6.2 Soll-Ist-Vergleich

Die Tabelle stellt die Pflichtanforderungen den tatsächlich umgesetzten Funktionen gegenüber. Fünf von sechs Punkten sind vollständig erfüllt. Bei den CRUD-Operationen gilt das nur teilweise: Lesen ist für alle sechs Tabellen umgesetzt, Löschen nur für Material und Kategorie; Anlegen und Bearbeiten fehlen noch. Das war eine bewusste Entscheidung (siehe [Kapitel 8.2](#)): Sowohl auf Datenbankebene über das Interface `FindDeleteDAO<T>` als auch auf Oberflächenebene über die Basisklasse `EntityCrud<T>` teilen sich alle sechs Entitäten denselben Ablauf, statt ihn sechsmal einzeln zu schreiben. Diese zweistufige Basis lässt sich später um Anlegen und Bearbeiten erweitern, ohne den bestehenden Code umzubauen.

Tabelle 8: Soll-Ist-Vergleich der Anforderungen

Anforderung	Umsetzung	Status
MySQL ≥ 5 Tabellen	6 Tabellen umgesetzt	erfüllt
CRUD-Operationen	Lesen (alle 6), Löschen (Material, Kategorie); Anlegen/Bearbeiten offen	teilweise
MVC-Architektur	Model / View / Controller + DAO	erfüllt
Such-/Filterfunktion	Suche nach Namen, Filter nach Station	erfüllt
Mindestbestand-Warnung	Rote Zeilenfärbung (RowFactory)	erfüllt
Buchungshistorie	Tabelle bestandsbewegungen	erfüllt

6.3 Abnahme und Übergabe

Die formale Abnahme erfolgt durch Frau Cramer zum Projektabschluss (KW 29, bis 17.07.2026). Grundlage sind die umgesetzten Pflichtanforderungen: lauffähige Anwendung, stabile Datenbankverbindung, korrekte Such- und Filterfunktion, sichtbare Mindestbestand-Warnung sowie eine vollständige Dokumentation. CRUD wurde im bewusst gewählten Umfang abgenommen: Lesen für alle sechs Tabellen, Löschen für Material und Kategorie (siehe [6.2](#)). Anlegen und Bearbeiten sind bewusst nicht Teil des Abgabestands. Übergeben wird die Projektdokumentation als PDF über Moodle, inklusive aller [Anhänge](#) A bis H. Der Quellcode liegt als Maven-Projekt vor und wird gesondert ebenfalls über Moodle als ZIP-Datei hochgeladen.

7 Dokumentation

7.1 Benutzerhandbuch

Für die Endnutzer entstand ein ausführliches Benutzerhandbuch. Es führt Schritt für Schritt durch Programmstart, Navigation und die einzelnen Funktionen – vom Anzeigen und Löschen

eines Materials über Suche und Filter bis zur Buchungshistorie – und schließt mit einem [FAQ-Teil](#) zur Fehlerbehebung. Das vollständige Handbuch steht in [Anhang C](#).

7.2 Entwicklerdokumentation

Damit sich das Projekt technisch nachvollziehen lässt, ist der Quellcode nach dem MVC- und DAO-Muster gegliedert und durchgehend kommentiert. Eine README erläutert den Aufbau des Projekts; gestartet wird die Anwendung über das Skript `start_lagerverwaltung.bat` per Doppelklick (siehe [Kapitel 8.2](#) und [Anhang C](#)). Zudem wurden Testdaten manuell in die Datenbank gepflegt. So findet jeder, der die Anwendung weiterentwickelt, einen schnellen Einstieg. Der gemeinsame Aufbau über die Schnittstelle `FindDeleteDAO<T>` und die Basisklasse `EntityCrud<T>` ist bewusst auf Erweiterung ausgelegt. Eine neue Funktion braucht nur eine weitere Methode im Vertrag, je DAO eine Umsetzung und den gemeinsamen Ablauf einmal in der Basisklasse. So lassen sich Anlegen und Bearbeiten später nachrüsten, ohne die bestehende Struktur umzubauen. Auch eine neue Tabelle folgt demselben Muster: eine Model-Klasse, eine DAO über die Schnittstelle und ein Reiter über `EntityCrud`. Über die Generics stellt der Compiler dabei sicher, dass sich jede Klasse an den Vertrag hält.

8 Fazit

8.1 Soll-Ist-Vergleich der Projektziele

Die Kernziele sind weitgehend erreicht. Drei der vier Pflichtenforderungen sind vollständig umgesetzt: die sechs Tabellen, die MVC-Architektur mit DAO-Schicht sowie Suche, Filter und Mindestbestand-Warnung laufen wie vorgesehen.

Beim CRUD wurden das Lesen für alle Entitäten und das Löschen für Material und Kategorie umgesetzt. Die verfügbare Zeit floss dann gezielt in eine saubere, gemeinsame generische Basis, statt in zusätzliche Formulare für das Anlegen und Bearbeiten (siehe [Kapitel 8.2](#)). Eine tragfähige, erweiterbare Grundlage hatte Vorrang. Sie legt den Grundstein für künftige Erweiterungen und lässt sich mit wenig Aufwand ausbauen. Insgesamt fällt das Ergebnis positiv aus.

Auch beim Zeitplan blieb das Projekt im Rahmen. Der eingeplante Puffer verschaffte in der Test- und Dokumentationsphase etwas Luft. Die folgende Tabelle stellt die geplanten den tatsächlichen Stunden je Phase gegenüber. Die größte Abweichung gab es in [Phase 3](#) (DAO & Model). Die generische DAO-Basis war ursprünglich nicht eingeplant und kam erst während der Umsetzung dazu. Das wurde in den anderen Phasen wieder aufgeholt.

Tabelle 9: Soll-Ist-Vergleich der Projektstunden

Phase		Soll (h)	Ist (h)	Differenz
1	Analyse & Planung	20	20	0
2	Datenbank	20	18	-2
3	DAO & Model	20	25	+5
4	UI-Grundfunktionen	40	41	+1
5	UI erweitert	25	24	-1
6	Test & Bugfixes	25	22	-3
7	Dokumentation	30	30	0
Gesamt		180	180	0

8.2 Lessons Learned

Das DAO-Muster, die Singleton-Verbindung und den Aufbau der Model-Klassen kannte der Autor schon aus dem Unterricht, aus dem Dino-Park-Beispiel von Frau Cramer. Die JavaFX-Themen musste er sich dagegen selbst beibringen. Dazu gehörten TableView mit ObservableList [\[5\]\[6\]](#), die gefilterte Bestandssicht, Dialoge [\[8\]](#), das TabPane und die Zeilenfärbung über eine RowFactory. Auf SQL-Ebene war die wichtigste Erkenntnis, den Bestand nicht zu speichern, sondern aus den Bewegungen zu berechnen. Im Umgang mit Fehlern hat sich ein festes Vorgehen bewährt: erst die Meldung lesen, dann die Ursache eingrenzen, am Ende die Lösung umsetzen.

Eine größere Umstrukturierung kam erst spät dazu und war ursprünglich nicht eingeplant. Dem Autor fiel während der Umsetzung auf, dass alle sechs DAO-Klassen immer wieder denselben Code brauchten. Kurz zuvor waren im Unterricht Interfaces durchgenommen worden. So entstand die Idee, die wiederkehrenden Zugriffe über eine gemeinsame Schnittstelle zu bündeln und das Projekt damit zu vereinfachen. Also wurden das Lesen und Löschen in der gemeinsamen Schnittstelle FindDeleteDAO<T> zusammengefasst und in jede DAO eingezogen. Neu war dabei der Typ-Platzhalter <T>, die Java-Generics. Am Anfang war nicht klar, warum in den spitzen Klammern ein Buchstabe statt eines richtigen Typs steht. Über ein Java-Tutorial [\[13\]](#) und eigenes Ausprobieren am Code wurde deutlich: <T> ist nur ein Platzhalter, der beim Implementieren festgelegt wird, zum Beispiel FindDeleteDAO<Material>. So merkt der Compiler Typfehler schon beim Übersetzen. Weil diese Umstrukturierung im ursprünglichen Plan nicht vorgesehen war, kostete sie zusätzliche Zeit. An dem Punkt stand die Entscheidung an: entweder schnell noch Formulare für das Anlegen und Bearbeiten zusammenbauen oder die Basis sauber fertig machen. Nach Rücksprache mit Frau Cramer entschied sich der Autor bewusst für die Basis und ließ dafür das

Anlegen und Bearbeiten weg. Eine solide, moderne Grundlage mit Schnittstelle und Generics war ihm wichtiger als zwei weitere Funktionen, die am Ende nur halb verstanden gewesen wären. Der gemeinsame Aufbau steht jetzt an einer Stelle, und der Compiler stellt sicher, dass sich jede Klasse daran hält. So lässt sich die Anwendung nach dem Projekt Schritt für Schritt weiterentwickeln, ohne etwas umbauen zu müssen.

Ähnlich verlief es später beim MainController. Mit der Zeit wurde er kaum noch zu überblicken. Wie schon bei den DAOs floss die Zeit hier in eine gemeinsame Basisklasse, `EntityCrud<T>`, statt in weitere Einzelfunktionen. Alle Reiter teilen sich seitdem den wiederkehrenden Ablauf in dieser Basisklasse. Der MainController reicht die Aktionen nur noch dorthin weiter. Das Interface mit Generics kam damit nicht nur bei den DAOs zum Einsatz, sondern nachträglich auch beim MainController. Funktional ist alles beim Alten geblieben, der Code ist aber kürzer und leichter zu lesen.

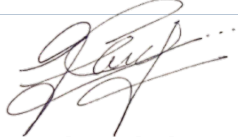
Auch der Programmstart selbst brachte eine Lektion. Die fertige Jar-Datei ließ sich unter Windows nicht einfach per Doppelklick starten: Maven war nicht im PATH eingetragen, danach fehlte auch Java selbst im PATH, und zuletzt öffnete Windows die Jar-Datei nur als Archiv, statt sie auszuführen. Jeder Schritt musste einzeln gefunden und geprüft werden, bevor klar war, woran es lag. Als pragmatische Lösung entstand ein kleines Start-Skript [\[16\]](#) (`start_lagerverwaltung.bat`), das die Jar-Datei zuverlässig über die Kommandozeile aufruft und bei einem Fehler das Fenster offenlässt, statt sich sofort zu schließen. Für einen produktiven Einsatz wäre ein richtiger Installer oder zumindest eine feste Dateizuordnung sauberer. Im Rahmen des Projekts reichte das Skript, um die Anwendung zuverlässig zu testen.

8.3 Ausblick

Der nächste Schritt ist das Anlegen und Bearbeiten von Datensätzen. Die Vorarbeit dafür ist gemacht: Die gemeinsame Basis über `FindDeleteDAO<T>` und `EntityCrud<T>` ist genau darauf ausgelegt (siehe [Kapitel 7.2](#)). Für einen produktiven Einsatz wären darüber hinaus einige Erweiterungen sinnvoll: eine Benutzerverwaltung mit Login und Rollen, eine Backup-Funktion und ein Export nach PDF und Excel. Für ein nächstes Projekt bleibt die Erkenntnis, die Testfälle früher und begleitend zur Entwicklung zu schreiben und die gemeinsame Basis für DAOs und Controller von Anfang an einzuplanen, statt sie erst nachträglich einzuziehen. Im gegebenen Zeitrahmen hatten diese Punkte bewusst keine Priorität.

Abgabeerklärung

Ich erkläre hiermit, dass ich diese Projektdokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Speyer, 17.07.2026	
Ort, Datum	Unterschrift (Sascha Schulz)

Quellenverzeichnis

Literaturquellen

- [4] SRH Berufliche Rehabilitation GmbH: DOIT-Leitfaden, Gruppe 2551, Heidelberg, 2024.

Internetquellen

- [1] Oracle Corporation: Java SE 17 Documentation. Oracle, 2021. URL:
<https://docs.oracle.com/en/java/javase/17/>
(Abruf: 04.05.2026)
- [2] Oracle / OpenJFX: JavaFX 21 API Documentation. 2021. URL:
<https://openjfx.io/javadoc/21/>
(Abruf: 04.05.2026)
- [3] Oracle / MySQL: MySQL 8.0 Reference Manual. 2023. URL:
<https://dev.mysql.com/doc/refman/8.0/en/>
(Abruf: 04.05.2026)
- [5] Oracle / OpenJFX: JavaFX 21 – TableView (API). URL:
<https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TableView.html>
(Abruf: 07.06.2026)
- [6] Oracle / OpenJFX: JavaFX 21 – ObservableList (API). URL:
<https://openjfx.io/javadoc/21/javafx.base/javafx/collections/ObservableList.html>
(Abruf: 07.06.2026)
- [7] Oracle / OpenJFX: JavaFX 21 – TabPane (API). URL:
<https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TabPane.html>
(Abruf: 07.06.2026)
- [8] Oracle / OpenJFX: JavaFX 21 – Dialog (API). URL:
<https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/Dialog.html>
(Abruf: 07.06.2026)
- [9] Oracle: Introduction to FXML (JavaFX-Tutorial). URL:
https://docs.oracle.com/javase/8/javafx/fxml-tutorial/get_started.htm
(Abruf: 07.06.2026)

- [10] Apache Software Foundation: Maven – Getting Started Guide. URL:
<https://maven.apache.org/guides/getting-started/>
(Abruf: 07.06.2026)
- [11] Oracle: MySQL Connector/J Developer Guide. URL:
<https://dev.mysql.com/doc/connector-j/en/>
(Abruf: 07.06.2026)
- [12] Oracle: Java SE 17 API – java.sql.PreparedStatement. URL:
<https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/PreparedStatement.html>
(Abruf: 16.07.2026)
- [13] Czeschla, Jörg: Was sind Generics? javabeginners.de. URL:
https://javabeginners.de/Klassen_und_Interfaces/Generics/Was_sind_Generics.php
(Abruf: 19.06.2026)
- [14] Oracle / MySQL: MySQL 8.0 Reference Manual – DELETE Statement. URL:
<https://dev.mysql.com/doc/refman/8.0/en/delete.html>
(Abruf: 16.06.2026)
- [15] Oracle / OpenJFX: JavaFX 21 – Alert (API). URL:
<https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/Alert.html>
(Abruf: 16.06.2026)
- [16] heise online: „Batch-Datei erstellen – so funktioniert’s“. URL:
<https://www.heise.de/tipps-tricks/Batch-Datei-erstellen-so-funktioniert-s-9971944.html>
(Abruf: 16.07.2026)

Anhang

Die folgende Tabelle gibt einen Überblick über die acht Anhänge dieser Dokumentation.

Anhang	Titel	Beschreibung
A	ER-Diagramm	Datenmodell der 6 Tabellen
B	SQL-Datenbankschema	CREATE-Statements aller Tabellen
C	Benutzerhandbuch	Vollständiges, detailliertes Handbuch
D	Wochenplan	Wochenplan_Sascha_Schulz.xlsx
E	Projektantrag	Projektantrag_Sascha_Schulz_v2.docx
F	Glossar	Fachbegriffe des Projekts (Java, JavaFX, JDBC, SQL)
G	Lastenheft	Anforderungen des Auftraggebers
H	Pflichtenheft	Technische Umsetzung der Anforderungen

Anhang A – ER-Diagramm

Das ER-Diagramm zeigt alle sechs Tabellen mit Entitäten, Attributen, Primär- und Fremdschlüsseln sowie den Kardinalitäten (1:m). Der Bestand wird nicht gespeichert, sondern aus bestandsbewegungen berechnet.

ER-Diagramm: Materialverwaltung Krankenhaus (6 Tabellen, 3NF)

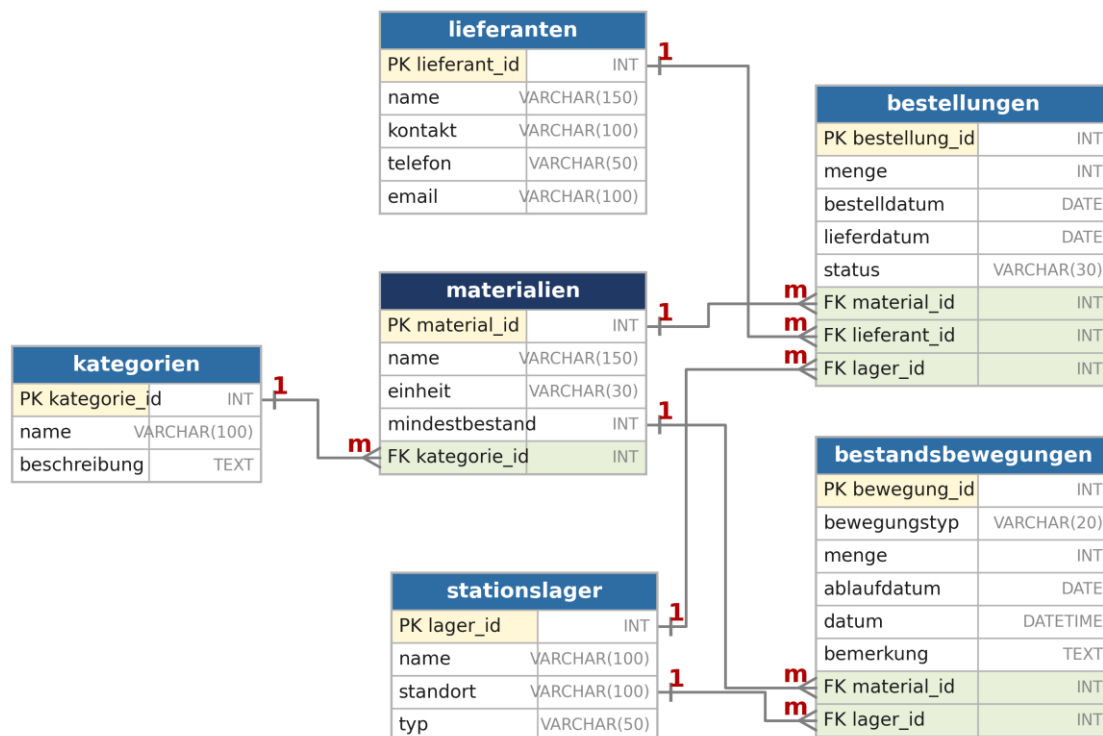


Abbildung 5: ER-Diagramm der Datenbank (Anhang A)

Anhang B – SQL-Datenbankschema

Datenbank DOIT. Die Testdaten wurden manuell in die SQL-Datenbank eingepflegt. Nachfolgend die CREATE-Statements aller sechs Tabellen.

```
CREATE DATABASE IF NOT EXISTS DOIT;
USE DOIT;

CREATE TABLE kategorien (
    kategorie_ID    INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    beschreibung    VARCHAR(100)
);

CREATE TABLE materialien (
    material_ID     INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(150) NOT NULL,
    einheit         VARCHAR(30) NOT NULL,
    mindestbestand  INT NOT NULL DEFAULT 0,
    kategorie_ID    INT NOT NULL,

    FOREIGN KEY (kategorie_ID) REFERENCES kategorien(kategorie_ID)
);

CREATE TABLE lieferanten (
    lieferant_ID    INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(150) NOT NULL,
    kontakt         VARCHAR(100) NOT NULL,
    telefon         VARCHAR(30) NOT NULL,
    email           VARCHAR(100) NOT NULL
);

CREATE TABLE stationslager (
    lager_ID        INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    standort        VARCHAR(100) NOT NULL,
    typ             VARCHAR(100) NOT NULL
);

CREATE TABLE bestellungen (
    bestellung_ID   INT AUTO_INCREMENT PRIMARY KEY,
    menge           INT NOT NULL,
    bestelldatum    DATE NOT NULL,
    lieferdatum     DATE NOT NULL,
    status          VARCHAR(30) NOT NULL,
    material_ID     INT NOT NULL,
    lieferant_ID    INT NOT NULL,
    lager_ID        INT NOT NULL,

    FOREIGN KEY (material_ID) REFERENCES
materialien(material_ID),
```

```

        FOREIGN KEY (lieferant_ID) REFERENCES
lieferanten(lieferant_ID),
        FOREIGN KEY (lager_ID)      REFERENCES stationslager(lager_ID)
);

CREATE TABLE bestandsbewegungen (
    bestandsbewegung_ID INT AUTO_INCREMENT PRIMARY KEY,
    bewegungstyp         VARCHAR(20) NOT NULL,
    menge                INT NOT NULL,
    ablaufdatum          DATE,
    datum               DATETIME NOT NULL,
    bemerkung            TEXT,
    material_ID          INT NOT NULL,
    lager_ID             INT NOT NULL,

    FOREIGN KEY (material_ID) REFERENCES materialien(material_ID),
    FOREIGN KEY (lager_ID)      REFERENCES stationslager(lager_ID)
);

```

Listing 7: Datenbankschema – CREATE-Statements aller sechs Tabellen

Anhang C – Benutzerhandbuch

Materialverwaltung für medizinisches Verbrauchsmaterial – Krankenhaus-Edition (JavaFX + MySQL). Version 1.0.

Zielgruppe: Pflege- und Stationspersonal sowie Verwaltung. Dieses Handbuch führt Schritt für Schritt durch alle Funktionen.

C.1 Systemvoraussetzungen

Damit die Anwendung läuft, müssen einige Voraussetzungen erfüllt sein. Die folgende Tabelle zeigt, welche Software und Hardware der Arbeitsplatz-Rechner mitbringen muss.

Tabelle 10: Systemvoraussetzungen

Komponente	Anforderung
Betriebssystem	Windows 10 oder 11 (64-Bit)
Java Runtime	Java 17 oder neuer (JRE/JDK)
Datenbankserver	MySQL 8.x – Dienst muss laufen
Datenbank	DOIT – muss angelegt sein
Auflösung / RAM	min. 1280 × 768; min. 4 GB RAM

C.2 Programmstart

So starten Sie die Anwendung:

1. Prüfen Sie, ob der MySQL-Dienst läuft (Windows-Dienste › „MySQL“ › Status „Wird ausgeführt“).
2. Starten Sie das Programm per Doppelklick auf start_lagerverwaltung.bat im Projektordner. Ein Fenster mit Statusmeldungen öffnet sich kurz, danach das Hauptfenster. Der Inhalt der Datei ist in [Listing 8](#) dokumentiert.

```
@echo off
cd /d "%~dp0target"
echo Starte Lagerverwaltung...
echo.
java -jar lagerverwaltung-1.0.0.jar
echo.
echo =====
echo Programm beendet oder Fehler oben sichtbar.
echo Fenster schliesst sich erst nach Tastendruck.
echo =====
pause
```

Listing 8: Start-Skript start_lagerverwaltung.bat

3. Das Hauptfenster öffnet sich und zeigt automatisch die Registerkarte „Bestandsübersicht“.

4. Erscheint die Meldung „Datenbankverbindung fehlgeschlagen“, starten Sie den MySQL-Dienst und prüfen die Zugangsdaten in DBConnection.java.

Das Fenster gliedert sich in die Tab-Leiste (oben), darunter je nach Registerkarte eine Schaltflächen-Leiste – in der Bestandsübersicht Suche, Stationsfilter, Suchen, Zurücksetzen und Aktualisieren, bei Materialien und Kategorien die Schaltfläche Löschen – und darunter die Tabelle.

Tabelle 11: Registerkarten und ihre Funktionen

Registerkarte	Funktion
Bestandsübersicht	Startansicht: Bestand je Material und Lager, Suche, Stationsfilter, Warnung
Materialien	Stammdaten der Materialien ansehen und löschen (Kern)
Kategorien	Materialgruppen ansehen und löschen
Stationslager	Stationen und Lager ansehen
Lieferanten	Lieferantenstammdaten (optional)
Bestellungen	Bestellungen einsehen und verfolgen (optional)
Bestandsbewegungen	Ein- und Ausgänge einsehen und nachvollziehen

C.3 Materialien ansehen

Öffnen Sie die Registerkarte „Materialien“. Die Tabelle zeigt alle Materialien mit Name, Einheit, Mindestbestand und Kategorie-ID. Die Materialien sind Stammdaten; der Bestand ergibt sich aus den Buchungen und wird in der Bestandsübersicht angezeigt.

C.4 Material löschen

Löschen: Zeile anklicken › „Löschen“ › Sicherheitsabfrage mit „OK“ bestätigen. Achtung: Gelöschte Datensätze lassen sich nicht wiederherstellen. Lässt sich ein Material nicht löschen, bestehen noch abhängige Bestellungen oder Buchungen, die zuerst entfernt werden müssen.

C.5 Suchen und Filtern

1. Öffnen Sie die Registerkarte „Bestandsübersicht“.
2. Tippen Sie im Suchfeld einen Materialnamen (z. B. Handschuh) und drücken Sie Enter oder die Schaltfläche „Suchen“. Groß-/Kleinschreibung spielt keine Rolle.
3. Wählen Sie im Dropdown „Station“ eine Station, um nur deren Bestände zu sehen.
4. Such- und Stationsfilter wirken zusammen. „Zurücksetzen“ leert beide Filter, „Aktualisieren“ lädt die Daten neu.

C.6 Mindestbestand-Warnung verstehen

In der Bestandsübersicht wird die gesamte Zeile rot eingefärbt, sobald der Bestand auf oder unter dem Mindestbestand liegt. Das ist das Signal, dieses Material dringend nachzubestellen. Die Färbung passt sich automatisch an, sobald durch eine Buchung wieder genug Bestand vorhanden ist. Zusätzlich zeigt die Status-Spalte den Text WARNUNG oder OK, und oben rechts zählt „Warnungen: n“, wie viele Zeilen aktuell betroffen sind.

C.7 Kategorien, Stationslager und Lieferanten

Diese Stammdaten werden auf den gleichnamigen Registerkarten angezeigt. Kategorien lassen sich zusätzlich löschen: Zeile anklicken › „Löschen“ › Sicherheitsabfrage bestätigen. Jedes Material muss einer Kategorie zugeordnet sein; eine Kategorie lässt sich nur löschen, wenn ihr kein Material mehr zugeordnet ist. Stationslager bilden Stationen und Lager ab (z. B. Station 1, OP, ICU, Zentrallager). Lieferanten (optional) speichern die Stammdaten der Bezugsfirmen und werden nur angezeigt.

C.8 Bestellungen ansehen

Öffnen Sie die Registerkarte „Bestellungen“. Die Tabelle zeigt je Bestellung Material-ID, Lieferant-ID, Lager-ID, Menge, Bestell- und Lieferdatum sowie den Status. Hinweis: Es werden aktuell nur die internen Nummern angezeigt, keine Klarnamen. Der Status beschreibt nur den Bestellvorgang und verändert den Bestand nicht.

C.9 Buchungen (Ein- und Ausgang)

Auf der Registerkarte Bestandsbewegungen werden alle Ein- und Ausgänge angezeigt und protokolliert. Jede Zeile zeigt Material-ID, Lager-ID, Bewegungstyp (EINGANG oder AUSGANG), Menge, Datum, Ablaufdatum und eine Bemerkung. Hinweis: Material und Lager werden aktuell nur als Nummer angezeigt.

Eine Umlagerung zwischen zwei Lagern steht als AUSGANG im Quelllager und EINGANG im Ziellager. So bleibt der Bestand lückenlos nachvollziehbar.

C.10 FAQ und Fehlerbehebung

Tabelle 12: Häufige Fragen und Fehlerbehebung

Frage / Meldung	Lösung
Programm startet nicht	Java 17 installiert? (java -version). MySQL-Dienst aktiv? Zugangsdaten prüfen.
Datenbankverbindung fehlgeschlagen	MySQL-Dienst in den Windows-Diensten starten.

Frage / Meldung	Lösung
Access denied for user	Falsches Passwort - Zugangsdaten in DBConnection.java prüfen.
Zeile ist rot markiert	Bestand kleiner oder gleich Mindestbestand - Material nachbestellen.
Material lässt sich nicht löschen	Erst abhängige Bestellungen/Buchungen entfernen.
Gelöschte Daten zurückholen?	Nicht möglich - mit Bedacht löschen.

Support: IT-Systemadministrator vor Ort oder Entwickler Sascha Schulz. Benutzerhandbuch Version 1.0 – SRH Berufliche Rehabilitation, WI-Ausbildung 2026.

Anhang D – Wochenplan

Der vollständige Wochenplan wurde wöchentlich, jeweils montags, als Datei Wochenplan_Sascha_Schulz.xlsx separat hochgeladen. Er enthält die wochenweise Planung der sieben Projektphasen über die sieben Projektwochen (04.05.2026 – 17.07.2026).

Die folgende Tabelle schlüsselt die sieben Projektphasen in ihre Teilaufgaben auf und zeigt den geplanten Aufwand je Aufgabe.

Tabelle 13: Detaillierte Aufschlüsselung der Projektphasen

Phase	Teilaufgabe	h
1 Analyse & Planung	Anforderungen & Ist-Analyse	8
	ER-Diagramm & Datenbankschema	6
	Wochenplan & Projektantrag	6
2 Datenbank	SQL-Skript & 6 Tabellen	10
	Testdaten einspielen	4
	JDBC-Verbindung (Singleton)	6
3 DAO & Model	Model-Klassen (6 POJOs)	6
	DAO-Klassen (Lesen/Löschen)	8
	Generisches Interface FindDeleteDAO<T>	6
4 UI-Grundfunktionen	Hauptfenster & TabPane	3
	Tabellen aller 6 Entitäten	14
	Basisklasse EntityCrud<T>	17
	Löschen & Dialoge	6
5 UI erweitert	Suche nach Namen	8
	Filter nach Station	8
	Mindestbestand-Warnung	9
6 Test & Bugfixes	Testfälle durchführen	12
	Fehler beheben	8
	Code-Review	5
7 Dokumentation	Benutzerhandbuch	8
	Projektdokumentation	21
	Abgabe & Moodle-Upload	1
Σ	Gesamt	180

Anhang E – Projektantrag

Der Projektantrag wurde vor Projektstart eingereicht und von der SRH genehmigt, bevor mit der Umsetzung begonnen wurde. Er dokumentiert die ursprüngliche Projektplanung mit Phasen und Stundenansätzen (180 h).

Anhang F – Glossar

Diese Übersicht enthält Fachbegriffe aus dem Projekt DOIT Lagerverwaltung.

Tabelle 14: Glossar der Fachbegriffe

Begriff	Kategorie	Einfache Erklärung
Alert	JavaFX UI	Kleines Meldungsfenster von JavaFX. Zeigt dem Nutzer eine Nachricht und wartet, bis er es schließt. Im Projekt für Fehler, Rückfragen und Hinweise genutzt (Dialoge.java).
AlertType	JavaFX UI	Bestimmt die Art des Alert-Fensters — also Icon und Standardtitel. Im Projekt kommen drei vor: ERROR (Fehler, rotes Symbol), CONFIRMATION (Ja/Nein-Rückfrage vorm Löschen) und INFORMATION (kurzer Hinweis, z. B. „erst eine Zeile markieren“).
ButtonType	JavaFX UI	Steht für einen Knopf in einem Alert-Fenster, z. B. ButtonType.OK oder ButtonType.CANCEL . Damit lässt sich nach dem Schließen prüfen, was der Nutzer geklickt hat. Im Projekt: Löschen läuft nur, wenn OK gedrückt wurde.
ComboBox	JavaFX UI	Ein Auswahlfeld zum Aufklappen (Dropdown). Der Nutzer wählt einen Wert aus einer Liste. Im Projekt filtert die comboBoxLager die Bestandsübersicht nach einem Lager.
HBox / VBox	JavaFX Layout	Zwei Anordnungs-Bausteine für die Oberfläche. HBox legt Elemente nebeneinander (horizontal), VBox untereinander (vertikal). Im Projekt in main.fxml : Suchfeld und Knöpfe in einer HBox , die ganze Maske in VBox .
ObservableList	JavaFX	Eine besondere Liste, die selbst Bescheid gibt, wenn sich ihr Inhalt ändert. Ist sie mit einer Tabelle verbunden, aktualisiert sich die Anzeige von allein. Erstellt mit FXCollections.observableArrayList() .
Parent	JavaFX Basisklasse	Oberbegriff für alle Bausteine, die andere Elemente enthalten können. Beim Start liefert loader.load() das oberste Element als Parent ; daraus wird die Scene (der Fensterinhalt) gebaut.
PropertyValueFactory	JavaFX TableView	Verbindet eine Tabellenspalte mit einem Feld des Datenobjekts. new PropertyValueFactory<>("materialName") sucht automatisch die Methode getMaterialName() . Der Name in den Klammern muss zum Getter passen.
TableRow	JavaFX TableView	Steht für eine einzelne Zeile in einer Tabelle. Über setRowFactory lässt sich jede Zeile eigen gestalten. Im

Begriff	Kategorie	Einfache Erklärung
		Projekt wird so eine Zeile rot eingefärbt, wenn der Bestand unter dem Mindestbestand liegt.
RowFactory	JavaFX TableView	Der „Zeilen-Bauplan“ einer Tabelle. Man übergibt ihn mit <code>setRowFactory</code> ; für jede Zeile entsteht daraus eine <code>TableRow</code> . In deren <code>updateItem()</code> wird entschieden, wie die Zeile aussieht (z. B. rote Färbung bei Warnung).
Enum	java.lang	Ein Aufzählungstyp mit einer festen Menge erlaubter Werte. Im Projekt: enum Bewegungstyp { EINGANG, AUSGANG }. In der Datenbank ist der Bewegungstyp als Text (VARCHAR) gespeichert; das Mapping auf das Java-Enum übernimmt die Anwendungslogik: <code>valueOf(...)</code> wandelt den Text in den Enum-Wert, <code>.name()</code> liefert ihn als Text zurück. Die Datenbank braucht dafür keine Änderung.
Generics	Java	Ein Platzhalter für einen Typ, in spitzen Klammern geschrieben. Er legt fest, welche Klasse erlaubt ist, z. B. <code>List<Material></code> oder <code>EntityCrud<T></code> . So merkt der Compiler Typfehler schon vorm Start. Das <code>T</code> wird erst beim Benutzen mit einer echten Klasse gefüllt.
DAO	Architektur	Data Access Object. Eine Klasse, die alle Datenbankzugriffe für eine Tabelle bündelt (z. B. <code>MaterialDAO</code> für <code>materialien</code>). Der Rest des Programms ruft nur Methoden wie <code>findAll()</code> oder <code>delete()</code> auf und sieht kein SQL.
JDBC	Datenbank	Java Database Connectivity. Die Standard-Schnittstelle, über die Java-Programme mit einer Datenbank sprechen. Im Projekt stellt der MySQL Connector über JDBC die Verbindung zur Datenbank her.
MVC	Architektur	Model-View-Controller. Aufteilung des Programms in Daten (Model), Anzeige (View, hier FXML) und Steuerung (Controller). Im Projekt ergänzt um die DAO-Schicht für die Datenbankzugriffe.
POJO	Java	Plain Old Java Object. Eine einfache Klasse nur mit Feldern, Konstruktoren und Gettern (z. B. <code>Material</code>). Sie spiegelt eine Tabellenzeile wider und enthält keine Programmlogik.
PreparedStatement	Datenbank	Vorbereitete SQL-Anweisung mit Platzhaltern (?). Die Werte werden getrennt vom SQL-Text übergeben; das schützt vor SQL-Injection. Im Projekt laufen alle Lese- und Löschzugriffe darüber.
Singleton	Architektur	Ein Entwurfsmuster, bei dem es von einer Klasse genau ein Objekt gibt. Im Projekt hält <code>DBConnection</code> genau eine

Begriff	Kategorie	Einfache Erklärung
		Datenbankverbindung, die alle DAO-Klassen gemeinsam nutzen.
SQL	Datenbank	Structured Query Language, die Abfragesprache der Datenbank. Im Projekt vor allem SELECT (Lesen) und DELETE (Löschen); der Bestand wird per SUM über die Bewegungen berechnet.

Anhang G – Lastenheft

Das Lastenheft beschreibt die Anforderungen aus Sicht des fiktiven Auftraggebers, der Krankenhausleitung. Es wurde in der Analysephase erstellt und war die Grundlage für das Pflichtenheft in [Anhang H](#).

G.1 Ausgangssituation

Die Verbrauchsmaterialien werden bisher von Hand über Listen und Tabellenkalkulationen verwaltet. Wer den Bestand einer Station braucht, muss ihn sich zusammensuchen. Eine automatische Warnung beim Unterschreiten des Mindestbestands fehlt, ebenso eine lückenlose Buchungshistorie. Bei der Pflege der Listen schleichen sich leicht Fehler ein.

G.2 Ziele

Das Personal soll jederzeit sehen, welche Bestände auf den einzelnen Stationen vorhanden sind. Wird ein Material knapp, soll die Anwendung darauf hinweisen. Jede Ein- und Ausbuchung soll protokolliert werden und nachvollziehbar bleiben. Nicht mehr benötigte Einträge sollen sich löschen lassen, ohne dass die Historie verloren geht.

G.3 Einsatzumgebung und Nutzer

Die Anwendung läuft lokal auf einem Arbeitsplatz-Rechner im Krankenhaus, ohne Internetanbindung. Nutzer sind Pflege- und Stationspersonal ohne technische Vorkenntnisse. Auftraggeber ist die Krankenhausleitung, die Anforderungen stellt und die Abnahme durchführt.

G.4 Pflichtenanforderungen

Die folgenden Anforderungen sind durch die Ausbildungsrichtlinie vorgegeben und müssen umgesetzt werden:

Tabelle 15: Pflichtenanforderungen laut Ausbildungsrichtlinie

Pflichtenanforderung	Beschreibung
Datenbankanbindung	MySQL-Datenbank mit mindestens 5 Tabellen
CRUD-Operationen	Erstellen, Lesen, Ändern, Löschen für alle Entitäten
MVC-Architektur	Klare Trennung von Model, View und Controller
Such- und Filterfunktion	Suche nach Materialname, Filter nach Station

Die Anforderung CRUD-Operationen wurde auf Lesen und Löschen begrenzt. Die dadurch frei gewordene Zeit floss in eine generische DAO-Basis (FindDeleteDAO<T>), die sich

später um Anlegen und Bearbeiten erweitern lässt. Die Abstimmung dazu ist in [Kapitel 8.2](#) beschrieben, Details in [Anhang H.1](#).

G.5 Zusatzanforderungen

Aus dem Krankenhausalltag kommen folgende Zusatzanforderungen:

Tabelle 16: Krankenhausspezifische Zusatzanforderungen

Zusatzanforderung	Beschreibung
Mindestbestand-Warnung	Visuelle Hervorhebung bei Unterschreitung
Buchungshistorie	Protokoll aller Ein- und Ausgänge
Stationsfilter	Anzeige nach Krankenhaus-Station
Benutzerfreundlichkeit	Intuitive Oberfläche für nicht-technisches Personal

G.6 Abnahmekriterien

Die Abnahme gilt als erfolgreich, wenn die definierten Testfälle ([Kapitel 6.1](#)) bestehen: Der Bestand wird korrekt aus den Bewegungen berechnet, Suche und Stationsfilter liefern die erwarteten Zeilen, Unterschreitungen des Mindestbestands sind auf einen Blick erkennbar, und das Löschen entfernt nur Datensätze, an denen keine Bestellungen oder Buchungen hängen.

Anhang H – Pflichtenheft

Das Pflichtenheft beschreibt, wie die Anforderungen aus dem Lastenheft ([Anhang G](#)) technisch umgesetzt werden. Es entstand am Ende der Entwurfsphase.

H.1 Zielbestimmung

Musskriterien sind die vier Pflichtanforderungen der Ausbildungsrichtlinie und die Mindestbestand-Warnung. Wunschkriterien sind die Übersichten für Lieferanten, Bestellungen und Bestandsbewegungen; sie wurden umgesetzt, sobald der Kern stand. Bewusst abgegrenzt sind das Anlegen und Bearbeiten von Datensätzen über die Oberfläche, eine Benutzerverwaltung, ein Betrieb im Netzwerk und Exportfunktionen ([Kapitel 8.3](#)). Das Anlegen und Bearbeiten wurde bewusst aus dem CRUD-Umfang herausgenommen; Benutzerverwaltung, Netzwerkbetrieb und Export waren bereits im Projektantrag als Einzelplatzlösung ohne diese Funktionen festgelegt.

H.2 Produktumgebung

Java 17 mit JavaFX 21 für die Oberfläche, MySQL 8 als Datenbank (Schema DOIT, Port 3324, lokal), JDBC mit Connector/J für den Zugriff, IntelliJ IDEA und Maven als Werkzeuge. Die Anwendung läuft als Einzelplatzlösung ohne Internetanbindung.

H.3 Produktfunktionen

F1 Bestandsübersicht: zeigt je Material und Lager den aktuellen Bestand. Der Bestand wird nicht gespeichert, sondern aus den Bewegungen berechnet – Summe der Eingänge minus Summe der Ausgänge.

F2 Suche und Filter: ein Suchfeld für den Materialnamen und eine Auswahl für die Station. Gefiltert wird beim Klick auf Suchen oder mit der Enter-Taste (`anzeigeFiltern()`).

F3 Mindestbestand-Warnung: liegt der Bestand auf oder unter dem Mindestbestand, färbt eine RowFactory die Zeile rot ein; ein Zähler über der Tabelle zeigt die Anzahl der Warnungen.

F4 Löschen: Material und Kategorie lassen sich nach einer Sicherheitsabfrage löschen (EntityCrud über `FindDeleteDAO<T>`). Hängen an einem Datensatz noch Bestellungen oder Buchungen, verhindert die Datenbank das Löschen und die Anwendung zeigt einen Hinweis.

F5 Übersichten: eigene Registerkarten zeigen Materialien, Kategorien, Stationslager, Lieferanten, Bestellungen und Bestandsbewegungen an.

H.4 Produktdaten

Sechs Tabellen in dritter Normalform: kategorien, materialien, stationslager, lieferanten, bestellungen und bestandsbewegungen. Das ER-Diagramm liegt in [Anhang A](#), das vollständige SQL-Schema in [Anhang B](#).

H.5 Benutzeroberfläche

Ein Hauptfenster mit sieben Registerkarten (TabPage); die Schaltflächen liegen oberhalb der Tabelle. Der Entwurf ist in [Kapitel 4.3](#) abgebildet, die Bedienung beschreibt das Benutzerhandbuch in [Anhang C](#).

H.6 Qualitätsanforderungen

Alle SQL-Zugriffe laufen über PreparedStatements, um SQL-Injection zu verhindern. Die Schichten sind nach MVC mit DAO-Schicht getrennt, die Datenbankverbindung ist als Singleton umgesetzt. Die Oberfläche soll ohne Einarbeitung bedienbar sein.

H.7 Zuordnung der Anforderungen

Die folgende Tabelle ordnet den zentralen Anforderungen aus dem Lastenheft die technische Lösung und das Kapitel der Dokumentation zu:

Tabelle 17: Zuordnung der Anforderungen zur technischen Umsetzung

Anforderung	Technische Umsetzung	Kapitel
Datenbank mit mind. 5 Tabellen	6 Tabellen in MySQL 8, dritte Normalform	4.1
CRUD-Funktionalität	Lesen für alle 6 Entitäten, Löschen für Material und Kategorie; Zugriff über FindDeleteDAO<T> mit PreparedStatements	4.2, 5.2
MVC-Architektur	View (FXML), Controller (EntityCrud<T>), Model mit DAO-Schicht	4.2
Suche und Filter	Suche nach Name, Filter nach Station (anzeigeFiltern())	4.3, 5.4
Mindestbestand-Warnung	RowFactory färbt betroffene Zeilen rot ein	4.3, 5.5